

Checklist

Self-Service Platform Onboarding

1. Prerequisites

- Confirm the Kind cluster is operational.
- Confirm ArgoCD is installed and working.
- Confirm the demo application is deployed.
- Confirm the Backstage developer portal is running.
- Confirm SSO is configured in Backstage.
- Confirm Keycloak has the platform realm configured.
- Confirm at least one Keycloak test user exists.
- Create a GitHub Personal Access Token with `repo` and `admin:org` scopes.
- Export the token as `GITHUB_TOKEN`.

2. Design the Onboarding API

- Use an API-first architecture rather than putting provisioning logic directly in Backstage.
- Create a `POST /api/v1/teams` endpoint.
- Require authentication and the `platform:teams:create` permission.
- Validate every request against an OpenAPI/JSON Schema.
- Require:
 - Team `name`
 - `display_name`
 - Team lead email
- Enforce lowercase alphanumeric team names with hyphens.
- Limit team names to 63 characters.
- Default the resource tier to `starter` when omitted.
- Track who submitted each onboarding request.
- Track when and why the request was submitted.
- Return provisioning status information to the caller.

- Keep the OpenAPI specification as the source of truth for clients and documentation.

3. Implement the Provisioning Workflow

- Validate the requester's permissions before touching infrastructure.
- Create the team's Kubernetes namespace.
- Apply team RBAC roles.
- Apply the correct resource quota.
- Create the team's source repository.
- Create the required Keycloak groups.
- Register the team in the Backstage catalog.
- Return the namespace, repository, and provisioning status.
- Make every provisioning step idempotent.
- Confirm that retrying a partially failed request does not duplicate resources.

4. Automate Namespace Provisioning

- Establish a consistent naming convention such as `team-{teamname}`.
- Use `team-{teamname}-{environment}` for environment-specific namespaces.
- Add labels for:
 - Team
 - Owner
 - Resource tier
 - Cost center
 - Managed-by
- Add useful integration annotations.
- Add the Backstage team/catalog URL annotation.
- Add a Slack channel annotation where applicable.
- Enable Istio sidecar injection when service-mesh enrollment is required.
- Automatically apply appropriate network policies.

5. Configure RBAC

- Create a `team-admin` role.
- Create a `team-developer` role.
- Create a `team-viewer` role.
- Give team admins full namespace-level control as intended.
- Allow developers to deploy and debug workloads.

- Give viewers read-only access.
- Prevent developers from deleting Kubernetes secrets.
- Reserve secret deletion for team-admin/platform-admin roles.
- Create RoleBindings for Keycloak/OIDC groups.
- Use consistent group names:
 - `{team}-admins`
 - `{team}-developers`
 - `{team}-viewers`
- Confirm team admins cannot modify `platform-*` namespaces.
- Confirm permissions cannot be escalated outside the team's namespace.

6. Configure Resource Quotas

- Define resource tiers such as:
 - Starter
 - Standard
 - Enterprise
- Define CPU request limits for each tier.
- Define memory request limits for each tier.
- Define CPU maximums.
- Define memory maximums.
- Limit persistent volume claims.
- Limit load balancer services.
- Limit the number of deployments where appropriate.
- Configure a Kubernetes `LimitRange`.
- Define default CPU and memory requests.
- Define default CPU and memory limits.
- Define maximum per-container CPU and memory.
- Connect quota alerts to the observability platform.
- Implement self-service quota upgrade requests.
- Add an approval workflow where governance is required.

7. Integrate Keycloak

- Use the Keycloak Admin REST API for group management.
- Create team groups during team provisioning.
- Create:
 - `{team}-admins`
 - `{team}-developers`

- `{team}-viewers`
- Add the team lead to the admins group.
- Add team members to the developers group.
- Match Keycloak group names to Kubernetes RoleBinding subjects.
- Include team membership in JWT claims where required.
- Implement periodic reconciliation between Keycloak and Kubernetes.
- Make Keycloak group creation idempotent.

8. Enable Self-Service Team Management

- Allow platform admins to create teams.
- Allow platform admins to assign the initial team admin.
- Allow team admins to add members.
- Allow team admins to remove members.
- Allow team admins to assign team roles.
- Allow team admins to request quota increases.
- Allow team admins to create sub-environments/sub-namespaces.
- Define team lifecycle states:
 - Active
 - Archived
 - Deleted
- Define inactivity rules for automatic archival.
- Define deletion rules for archived teams.

9. Build Project Templates

- Treat a project as a complete developer unit rather than only a repository or namespace.
- Provision the following together:
 - Source repository
 - Deployment namespace(s)
 - CI/CD pipeline
 - Backstage catalog entry
 - Documentation scaffolding
- Define project naming conventions that flow into downstream resources.
- Organize templates by archetype, for example:
 - Backend service
 - Frontend application
 - Data pipeline
 - ML model

- Add language/framework-specific variants.
- Store templates in version-controlled Git repositories.
- Include template metadata and supported platform features.
- Provide a preview of resources before execution.

10. Configure Backstage Scaffold

- Create a Backstage software template.
- Collect the project/service name.
- Collect the owning team.
- Collect the template type.
- Provide resource tier selection.
- Default the tier to `starter`.
- Default the environment to `dev`.
- Default repository visibility appropriately.
- Validate project-name uniqueness.
- Validate team membership.
- Validate quota availability.
- Require a cost center for production environments where applicable.
- Require GPU quota approval for ML templates where applicable.
- Call the onboarding API from the Backstage workflow.
- Publish the GitHub repository.
- Surface a link to the source repository after completion.
- Surface a link to the Backstage catalog entry after completion.

11. Handle Failures Safely

- Treat every provisioning operation as potentially fallible.
- Pre-check cluster capacity before provisioning.
- Return `409 Conflict` when requested quota exceeds available capacity.
- Return an appropriate `503` for Kubernetes infrastructure failures.
- Handle GitHub `429` rate-limit responses.
- Handle Kubernetes API `503` responses.
- Implement exponential backoff.
- Add jitter to retry behavior.
- Record failed attempts in the audit trail.
- Verify retries are safe.
- Prevent duplicate namespace creation.
- Prevent duplicate identity groups.

- Prevent duplicate repositories/catalog entities.
- Define rollback behavior for project-level partial failures.

12. Add Observability and Auditability

- Expose Prometheus metrics.
- Produce structured logs.
- Produce distributed traces where applicable.
- Audit successful provisioning operations.
- Audit failed provisioning operations.
- Record the requesting user.
- Record provisioning timestamps.
- Record the requested purpose/context.
- Record retry attempts.
- Track time-to-first-deploy.
- Track onboarding ticket count.
- Track onboarding abandonment rate.

13. Test the End-to-End Developer Experience

- Open the developer portal as a test user.
- Select a project template.
- Enter the service name.
- Select the owning team.
- Select the resource tier.
- Submit the request.
- Confirm the Git repository is created.
- Confirm the namespace is created.
- Confirm RBAC is applied correctly.
- Confirm quota and LimitRange resources are applied.
- Confirm Keycloak groups are created.
- Confirm the Backstage catalog entity is registered.
- Confirm the CI/CD pipeline is configured.
- Confirm Kubernetes access works.
- Confirm the first deployment is ready to trigger.
- Verify the complete workflow can finish in approximately five minutes.

14. Production Readiness

- Document the API with OpenAPI/Swagger UI.
- Establish API versioning rules.
- Use URL-based versions for breaking changes.
- Define a deprecation policy.
- Announce deprecations before removing versions.
- Generate client SDKs where useful.
- Secure all secrets appropriately.
- Validate authorization before provisioning.
- Confirm every provisioning operation is idempotent.
- Load-test the onboarding API.
- Test GitHub API rate-limit behavior.
- Test Kubernetes API failure scenarios.
- Test quota exhaustion.
- Test partial provisioning failures.
- Test permission-escalation attempts.
- Document rollback and recovery procedures.

15. Future Maturity Review

- Assess whether the custom API is becoming difficult to maintain.
- Evaluate Argo Workflows for persistent multi-step orchestration.
- Evaluate Tekton for Kubernetes-native workflows.
- Evaluate Crossplane for declarative infrastructure provisioning.
- Evaluate Kratix for promise-based platform capabilities.
- Evaluate a commercial orchestrator if reducing maintenance burden becomes a priority.
- Compare learning curve, flexibility, maintenance, GitOps alignment, complexity, time-to-value, and cost before migrating.
- Preserve the API-first and self-service principles regardless of the orchestration technology selected.